

Lab Report: Processor Design and Implementation

Hassan Akhter(ha10609)/Arham Hussain (sh10331)

1. Introduction

In this Project, we designed and implemented a simple RISC-V processor by developing individual modules and integrating them into a complete system. The key components include the Register File, Arithmetic Logic Unit (ALU), Instruction Memory, Immediate Generator, and ALU Control Unit, and the DATA Memory all available in the code directory.

A brief Description of the most important modules:

Instruction Memory: A read-only memory block that stores machine code and outputs a 32-bit instruction based on the current Program Counter (PC) address.

Register File: Fast internal storage containing 32 general-purpose registers used to hold active data for the processor's operations.

Immediate Generator: A module that extracts specific bits from an instruction and rearranges them into a full 32-bit signed integer for use in calculations.

ALU Control Unit: A combinational logic block that decodes the instruction's funct/opcode bits to tell the ALU exactly which operation (add, sub, and, etc.) to perform.

Data Memory: A read/write memory space used to store and retrieve long-term data via Load and Store instructions.

Each module was first designed and verified independently. Subsequently, In Lab 11 all modules were integrated to form a functional processor capable of executing assembly instructions in Task A of the Project. In Task B we extended functionality of the TopProcessor by implementing 3 additional instruction, which were BLT(Branch if less than) for comparisons, JAL (Jump And Link) for function calls, and LUI(Load upper Immediate) for large number operations.

Link to Repo: <https://github.com/hAkhter89/CA-Labs-T1/tree/main/projcodeDirectory>

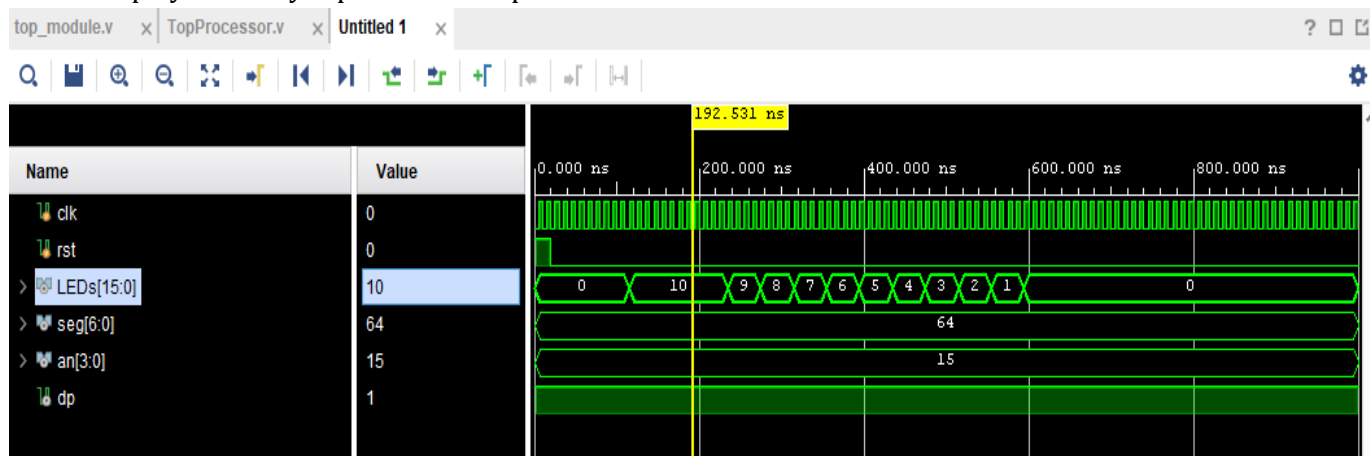
2. Task A: Countdown Assembly Program

In Task A, we created a simple countdown program in Assembly, to countdown from 10 and display the result in a memory location that we wired to the FPGA leds so that we can see the countdown happening physically. However, to accomplish this we had to create another module to slow down the processor so that we could see what was happening in the FPGA.

1. Developed and verified all required RTL modules including ALU, Register File, Instruction Memory, and Immediate Generator.
2. Integrated all modules into a top-level processor design.

3. Created a testbench to simulate the processor behavior.
4. Loaded the assembly program into the instruction memory.
5. Load constrains to make use of the LEDs to display memory location and ALUResults along with the multiplexed seven segment display.
6. Created a testbench to verify that out values were decrementing in the LED simulated memory location and that the processor was working.
7. Created a clk_divider with a 1hz clock and wired it to the Processor to see the countdown results happening on the FPGA, the solution and worked and the processor was slowed down to execute about 1 instruction per second and we could see the results via LEDs in realtime.

The processor successfully executed the assembly instructions, and the outputs matched the expected behavior, with the LEDs(memory location 8) showing the countdown being executed properly from 10, all the way to 0. the seg counter stays at 64 to represent all 0's on the display as it only represent the top 16 bits of the data.



3. Task B: New Instrurctions (JAL, BLT, LUI)

Instruction Description:

3 new instructions, BLT, JAL and LUI, were implemented. These instructions are used to:

- 1) BLT: compare 2 registers and branch if the value in register A is less than in register B (useful for bubblesort where we have to compare two successive values and swap them if they are not in order)
- 2) JAL: works like a function call, jumps to the required label and saves the return address (of the current label).
- 3) LUI: loads a large value, takes a 20bit value and stores in in the upper 20bits of the 32-bit register, this is useful because the immediate value in the instruction cannot be 32bits (we still need space for the opcode and funct7/funct3). Therefore we need LUI to work with a bigger value.

Datapath Modifications:

- Several Modifications were made in ALU and immGen to account for these new instruction (all changes are marked TASK2 Addition in the relevant files)
- Added a new JAL MUX to the datapath, and new data_path changes for immGen to handle additional B-type, J-type and L-type instructions.
- Ensured the new changes did not break anything by testing previous testbenches with the new changes.

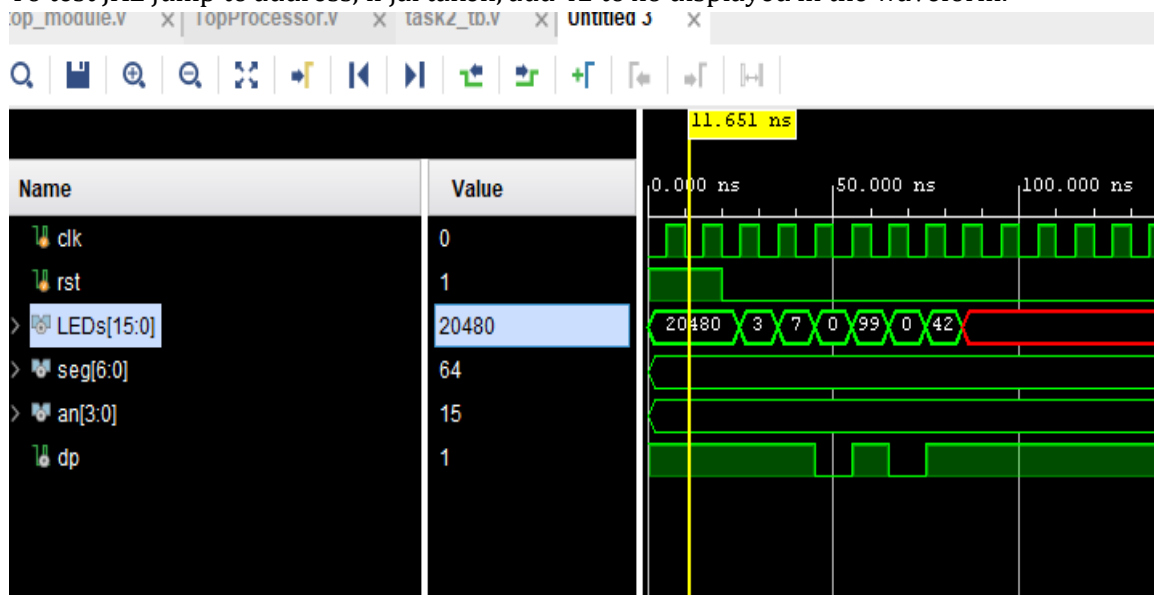
Control Path Changes:

- Modified the control units to ensure compatibility, changes in MainControl, ALUControl, and new control signals like JAL, BLT_taken.
- Added new control signal conditions, changed PCSrc to account for JAL, BLT_taken to ensure branching compatibility with the new changes.
- Updated decoding logic in the ALUControl and MainControl.

Verification:

Created a testbench to test the new additions, used LEDs and 7segment to view the ALUResult values this time and compare with expected results, made custom testcases in the testbench to verify proper function. Initially the value 20480 is loaded via LUI, then blt is tested $3 < 7$, 99 is added to x4 -> all displayed in the waveform.

To test JAL jump to address, if jal taken, add 42 to x6 displayed in the waveform.



```

=== TASK 2 DEBUG ===
PC | inst | ALUOp | ALUctrl | ALUSrc | imm | ALU_B | ALUResult | writeData | writeData_final | RegWrite
-----
4 | 00300113 | 10 | 0010 | 1 | 00000003 | 00000003 | 00000003 | 00000003 | 00000003 | 1
8 | 00700193 | 10 | 0010 | 1 | 00000007 | 00000007 | 00000007 | 00000007 | 00000007 | 1
12 | 0031c463 | 01 | 0110 | 0 | 00000008 | 00000007 | 00000000 | 00000000 | 00000000 | 0
20 | 06300213 | 10 | 0010 | 1 | 00000063 | 00000063 | 00000063 | 00000063 | 00000063 | 1
24 | 008002ef | 00 | 0010 | 0 | 00000008 | 00000000 | 00000000 | 00000000 | 0000001c | 1
32 | 02a00313 | 10 | 0010 | 1 | 0000002a | 0000002a | 0000002a | 0000002a | 0000002a | 1
36 | xxxxxxxx | 00 | 0010 | 0 | 00000000 | xxxxxxxx | xxxxxxxx | xxxxxxxx | xxxxxxxx | 0
40 | xxxxxxxx | 00 | 0010 | 0 | 00000000 | xxxxxxxx | xxxxxxxx | xxxxxxxx | xxxxxxxx | 0

=== RESULTS ===
LUI PASS: x1 = 0x12345000 (expected 0x12345000)
BLT PASS: x4 = 99 (expected 99)
JAL PASS: x6 = 42 (expected 42)

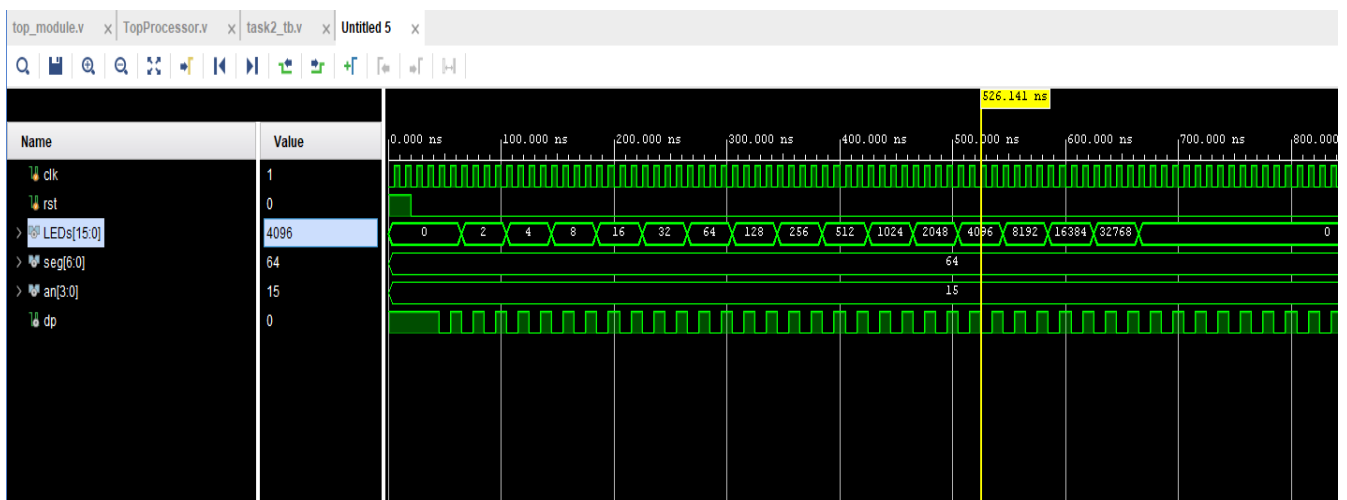
```

4. Task C: Simple Calculator Program

Created a simple calculator program, takes a number via assembly and shifts up to it (multiplication) until the number is exceeded. Uses BLT to compare the base number (initially 1) and target number (loaded via LUI). It also uses JAL to jump to a separate display subroutine which displays the result (new multiplied number) to LEDs and 7segment display. The following steps were taken to make this work:

8. Loaded the updated assembly program into instruction memory. (task3.mem)
9. Used the modified processor design from Task B.
10. Updated constraints and ran simulation. (task3_tb)
11. Verified execution through waveform analysis (mem location values to LEDs).

Waveform results of the LEDs memory location to check whether multiplication was taking place, in the waveform, the value is being multiplied by 2 until it exceeds the target value, the final value recorded is 32768. The BLT, AND JAL branching is displayed by the dp wire.



5. Conclusion

In this lab, we successfully designed and implemented a RISC-V processor by integrating multiple modules. We executed assembly programs and extended the processor by adding 3 new instructions (JAL, BLT, LUI). Made modifications to the datapath and control unit and verified them through simulation. Overall, the lab provided a clear understanding of processor design, instruction execution, and all the tasks were demonstrated to the Instructor on FPGA via a clock divided `slow_clk` which slowed down the processor to 1 instruction/sec for human verification.